

Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering

Term paper submitted in partial fulfillment of the requirements for the Master's Degree in Information Technology (MIT), Third Semester

TRIBHUVAN UNIVERSITY

APRIL 2026

Melina Giri | Roll No: 8125018

Central Department of Computer Science and Information Technology,
Kathmandu, Nepal



Introduction & Problem Statement

The Challenge

- Keyword search returns many documents, not direct answers
- Growing data needs context-aware retrieval and generation

The RAG Solution

- Retrieve relevant document sections first
- Use them as context for LLM answer generation
- Helps overcome limits of standalone models

Core Problem

- Too few sections can miss key information
- Too many sections add redundancy and latency
- This study tests these trade-offs on the Vectara Open RAG Benchmark

Objectives

Build the Pipeline

- Document processing
- Embeddings
- Retrieval and answer generation

Measure Retrieval

- Hit@k, MRR, and nDCG
- $k = 3, 5, 10$, and 20
- 3,045 benchmark queries

Compare Configurations

- Top-3 retrieval
- Top-10 retrieval
- Optimized top-10 on 60 questions
- Lexical F1 used for comparison

Optimize Context

- Context size
- Generation latency
- Statistical significance of F1 differences

Methodology & RAG Pipeline

1

Document Load & Embed Gen

Loads pre-segmented benchmark sections and generates dense embeddings.

2

Vector Retrieve

Retrieves relevant sections with cosine similarity.

3

Context Optimize

Optionally removes redundant context using Jaccard overlap ≥ 0.70 .

4

Response Generation

Generates answers with a locally executed language model.

Dataset & Implementation Tools

Study Setup

- **Dataset:** Vectara Open RAG Benchmark
- **1,000 PDF papers** and **3,045 QA pairs**
- Abstractive and extractive questions
- Source types: text, text-image, text-table, text-table-image
- **Embeddings:** all-MiniLM-L12-v2, 384-dim
- **Retrieval depths:** k=3, 5, 10, and 20
- **Configurations:** R0_k3, R1_k10, R2_k10_optimized
- **Model:** Qwen2.5-3B via Ollama

Implementation Tools

- **LangChain** — pipeline orchestration
- **Sentence-Transformers** — dense embeddings
- **Ollama** — local LLM inference
- **SciPy** — statistical tests

Retrieval Performance Results

Key Findings (3,045 Queries)

- Higher retrieval depth improves all three metrics.
- More depth increases relevant section recall and ranking quality.

Hit@k

61.18% → 82.33%

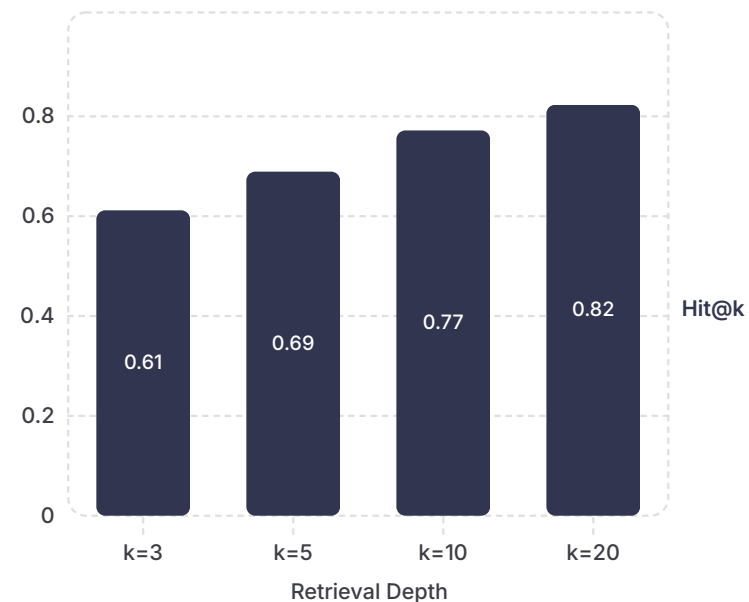
k=3 to k=20

MRR

0.5061 → 0.5385

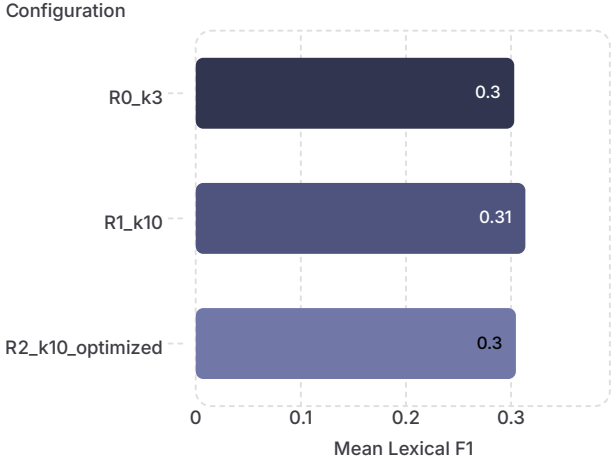
nDCG

0.5333 → 0.6051



Generation Performance: Three Configurations

Metric	R0_k3	R1_k10	R2_k10_optimized
Mean Lexical F1	0.3030	0.3136	0.3046
Latency (s)	185.600	16.958	12.098
IDK Rate	23.33%	23.33%	25.00%
Context (chars)	7,014	8,000	7,862



❏ R1_k10 delivers the best F1 (0.3136), while R2_k10_optimized is fastest at 12.098s; Latency reduced by 28.7% vs. R1_k10, and all 180 generations completed without errors.

Statistical Significance & Key Trade-offs

Wilcoxon Signed-Rank Test (n=60 paired questions)

Comparison	$\Delta F1$	p-value	Significant?
R0_k3 → R1_k10	+0.0106	0.263	No
R1_k10 → R2_k10_optimized	-0.0090	0.005	Yes
R0_k3 → R2_k10_optimized	+0.0016	0.942	No

- Context optimization (R2_k10_optimized) reduced latency by ~4.86s.
- It also caused a **statistically significant** drop in lexical F1 vs. R1_k10.
- Trade-off: higher efficiency, lower word-overlap answer quality.

Conclusion & Future Recommendations

Validated

- Retrieval depth improves coverage
- Hit@20: 82.33%
- Context optimization lowers latency
- Small but statistically significant F1 decrease

Limitations

- Only 60 generation questions
- Lexical F1 measures word overlap, not semantics
- General-domain documents only
- R0_k3's high latency (185.6s) may be a runtime artifact, not isolated

Future Work

- Expand the generation dataset
- Use semantic evaluation: BERTScore, LLM-as-judge
- Test stronger embedding and LLM models
- Deploy in real-world applications

 **Keywords:** Retrieval-Augmented Generation · Document Question Answering · Dense Retrieval · Sentence Embeddings · Vector Database · Context Optimization · Open RAG Benchmark · Qwen2.5

Thank You

This research evaluated different configurations for Retrieval-Augmented Generation in Document Question Answering, demonstrating the trade-off between retrieval depth, context optimization, and model performance. We hope these findings contribute to building more efficient and accurate RAG systems.

Melina Giri | Roll No: 8125018

Central Department of Computer Science and Information Technology, Tribhuvan University